

Cricket Analytics Data Pipeline — Project Documentation

Built on Snowflake | Landing → Raw → Clean → Consumption Architecture

1. What Was the Project?

This project builds an end-to-end data engineering pipeline on **Snowflake** that ingests raw cricket match data (in nested JSON format, ~2,411 files) and transforms it into a clean, query-ready **star schema** for analytics and reporting.

The pipeline follows a classic **four-layer (medallion-style) architecture**:

Layer	Schema	Purpose
Landing	<code>cricket.land</code>	Raw file storage — JSON files are staged exactly as received, no transformation
Raw	<code>cricket.raw</code>	Structured ingestion — JSON is loaded into a table with <code>VARIANT</code> / <code>OBJECT</code> / <code>ARRAY</code> columns, preserving the nested structure
Clean	<code>cricket.clean</code>	Parsed and validated — nested JSON is flattened into proper relational tables (<code>match_detail_clean</code> , <code>player_clean_tbl</code> , <code>delivery_clean_tbl</code>) with data types enforced, nulls checked, and keys defined
Consumption	<code>cricket.consumption</code>	Business-ready — a star schema (dimension + fact tables) built for reporting tools (Tableau, Power BI, SQL analytics)

Each source file represents one cricket match, broken into three logical sections:

`meta` — file/schema version metadata

`info` — match-level details (teams, venue, toss, result, dates, season, etc.)

`innings` — ball-by-ball delivery data (overs, deliveries, runs, extras, wickets)

2. Why This Project? Business Requirement

Raw cricket data (such as the open-source Cricsheet JSON dataset) is delivered as deeply nested, semi-structured JSON — great for storage, unusable for direct reporting. Analysts and stakeholders need to

answer questions like:

Which batter scored the most runs in a given match or series?

What is a team's win/loss record by venue, opponent, or toss decision?

How do run rates, extras, and wickets trend across overs?

Which venues favor bowling first vs. batting first?

None of this is answerable directly against nested JSON with acceptable performance or usability. The **business requirement** was therefore to:

Ingest raw JSON reliably and repeatably (handle hundreds/thousands of files, track file lineage).

Structure the nested data into clean, typed, relational tables.

Model the data into a dimensional (star) schema so business users can run fast, simple aggregations (SUM runs, COUNT wickets, GROUP BY team/venue/date) without needing to understand JSON.

Validate the data at each stage to ensure trustworthy numbers before they reach a dashboard.

In short: turn "data that exists" into "data that's usable" — the core job of any data engineering pipeline.

3. Why Analyze Cricket Data Specifically?

Cricket was chosen as the subject domain because it's an ideal training ground for data engineering skills:

Deeply nested, real-world JSON — matches → innings → overs → deliveries → extras/wickets. This mirrors the kind of nested structures seen in real production systems (e-commerce orders, IoT events, clickstreams), making it excellent practice for `VARIANT`, `LATERAL FLATTEN`, and semi-structured data handling.

Naturally dimensional data — matches have dates, venues, teams, players, and outcomes, which map cleanly onto a **star schema** (fact table = deliveries/match events, dimensions = date, team, player, venue, match type).

High data volume at the grain of a single ball — a single match can have 250–600+ delivery records, giving a realistic volume to test performance, joins, and aggregation logic.

Freely available, well-documented dataset — the Cricsheet-style JSON format is a widely used public dataset for practicing exactly this kind of pipeline.

Business-relevant analytics — sports analytics (player performance, team strategy, venue trends) is a real, active industry use case, not just a toy example.

4. Architecture Deep Dive — Stage by Stage

4.1 Landing Layer (`cricket.land`)

Purpose: Land the raw files exactly as they arrive, with no transformation.

Key steps performed:

Created the database and all four schemas (`land`, `raw`, `clean`, `consumption`).

Created a **file format** (`my_json_format`) with `TYPE = JSON` and `STRIP_OUTER_ARRAY = TRUE`, since each file's outer array needed to be removed to expose individual JSON objects.

Created an internal **stage** (`my_stg`) to hold the uploaded `.json.gz` files.

Because Snowflake's web UI stage upload is capped around 250 MB, the actual file upload for thousands of files was done via **SnowSQL** (the CLI client), not the web UI.

Used `SELECT $1:meta, $1:info, $1:innings ... FROM @my_stg (FILE_FORMAT => 'my_json_format')` with `METADATA$FILENAME`, `METADATA$FILE_ROW_NUMBER`, etc., to preview and confirm the structure before loading further.

4.2 Raw Layer (`cricket.raw`)

Purpose: Load the JSON into a structured (but still semi-structured) table, preserving full fidelity of the source and adding lineage columns.

Created `match_raw_tbl` with columns:

`meta OBJECT`, `info VARIANT`, `innings ARRAY` — the three JSON sections, kept intact for flexible downstream parsing.

`stg_file_name`, `stg_file_row_number`, `stg_file_hashkey`, `stg_modified_ts` — lineage/audit columns so every row can be traced back to its source file.

Loaded ~2,411 JSON files using `COPY INTO ... ON_ERROR = CONTINUE`, so a handful of malformed files don't block the entire batch load.

Verified load success with a row count check (`SELECT COUNT(*)`) against the known source file count.

4.3 Clean Layer (`cricket.clean`)

Purpose: Flatten the nested JSON into proper relational tables — this is where most of the transformation logic lives.

Three core clean tables were built:

a) `match_detail_clean` — one row per match, extracted from the `info` VARIANT column using `:` path notation and `::` type casts, e.g.:

```
sql
info:match_type_number::int as match_type_number,
info:dates[0]::date as event_date,
info:teams[0]::text as first_team,
info:teams[1]::text as second_team
```

Includes derived/business logic columns such as `match_stage` (event round/number, with a fallback to `'NA'`), `match_result` (Result Declared / Tie / No Result), and `winner` (defaulted to `'NA'` when there is no winner). A **primary key** constraint was added on `match_type_number`.

b) `player_clean_tbl` — one row per player per team per match, built using `LATERAL FLATTEN` (explained in Section 5) to explode the nested `players` object (keyed by country, valued as an array of player names) into rows. `NOT NULL` constraints were added on the key columns, and a **foreign key** back to `match_detail_clean` was defined.

c) `delivery_clean_tbl` — the ball-by-ball fact-level detail, built by **chaining five nested `LATERAL FLATTEN` calls** to walk down: `innings → overs → deliveries → extras → wickets`. This produces one row per delivery (with extra rows where a delivery has multiple extras/wicket entries), capturing bowler, batter, non-striker, runs, extras, extra type, and dismissal details. `NOT NULL` and foreign-key constraints were applied here too.

4.4 Data Validation

Before moving to consumption, targeted validation queries were run against a known match (India vs. England, 2023-10-29, `match_type_number = 4686`) to sanity-check the pipeline:

Pulled the full match detail row to confirm metadata (teams, venue, result) looked correct.

Aggregated runs **by batter and country** and compared against known scorecards.

Joined `delivery_clean_tbl` to `match_detail_clean` to compute **total match runs (runs + extras) by venue and teams**, cross-checking against the actual match total.

This step is what actually proves the flattening logic didn't duplicate or drop delivery records — a very easy mistake to make with multiple chained `LATERAL FLATTEN` joins.

4.5 Consumption Layer (`cricket.consumption`) — Star Schema

Purpose: Model the clean data into a dimensional schema optimized for reporting.

Dimension tables:

`date_dim` — full calendar attributes (day, month, year, quarter, day name, `is_weekend` flag), generated from a date-spine table (`date_rnage01`) and `ROW_NUMBER()`.

`team_dim` — distinct team names, sourced with `SELECT DISTINCT ... UNION ALL` across `first_team`/`second_team` from `match_detail_clean`.

`player_dim` — distinct players, linked to `team_dim` via foreign key.

`venue_dim` — venue attributes (city, country, capacity, floodlights, coordinates, etc.).

`match_type_dim` — match format (e.g., ODI, T20, Test).

`referee_dim` — referee/official details.

Fact tables:

`match_fact` — one row per match, at the grain of the whole match: overs played, balls bowled, extras, fours/sixes, total score, and wickets lost — **for each team separately (Team A / Team B)** — plus toss winner, toss decision, and match winner. Populated with a single `INSERT` that joins `match_detail_clean` to every dimension and aggregates `delivery_clean_tbl` with conditional (`CASE WHEN`) aggregation.

`delivery_fact` — one row per delivery, at ball-by-ball grain: match, team, bowler, batter, non-striker (all resolved to surrogate keys via joins to `player_dim`/`team_dim`), over, runs, extras, and dismissal info.

This star schema (facts surrounded by conformed dimensions) is what lets a BI tool answer "runs by venue by year" or "wickets by bowler by match type" with a simple join-and-aggregate, instead of a nested-JSON traversal.

5. What Is `LATERAL` and `FLATTEN`?

This is one of the most important — and trickiest — techniques used throughout the Clean layer, so it deserves its own explanation.

`FLATTEN` is a Snowflake table function that takes a semi-structured value (an `ARRAY`, `OBJECT`, or `VARIANT`) and **explodes it into rows** — one row per element. For each element it returns useful columns like:

`VALUE` — the actual element's contents

`KEY` — the key name (for objects)

`PATH` — the full path to the element

`INDEX` — the position (for arrays)

`LATERAL` is what allows `FLATTEN` (or any table function) to be **correlated to the row it's operating on**. A normal join can't reference columns from a table function that itself depends on a column from the same query — `LATERAL` solves this by evaluating the table function "row by row," letting each `FLATTEN` call see the current row's value as its input.

Why chaining is needed: cricket delivery data is nested multiple levels deep — `innings` → `overs` → `deliveries` → `extras` / `wickets`. A single `FLATTEN` can only explode one level. To fully unnest all the way down to "one row per delivery," the pipeline chains multiple `LATERAL FLATTEN` calls, each one flattening the *output* of the previous one:

```
❏
sql
from raw.match_raw_tbl m,
lateral flatten (input => m.innings) i,      -- match → innings
lateral flatten (input => i.value:overs) o,   -- innings → overs
lateral flatten (input => o.value:deliveries) d, -- overs → deliveries
lateral flatten (input => d.value:extras, outer => True) e, -- delivery → extras
lateral flatten (input => d.value:wickets, outer => True) w -- delivery → wickets
```

A few important details from the actual implementation:

`outer => True` was used on the `extras` and `wickets` flattens. Without this, a `FLATTEN` performs an *inner*-join-like behavior — if a delivery has no extras or no wicket (the vast majority don't), that row would be **dropped entirely**. `outer => True` preserves the delivery row with `NULL`s when there's nothing to flatten, similar to a `LEFT JOIN`.

The **player extraction** (`[player_clean_tbl]`) uses a simpler two-level flatten: first over the `[players]` object (keyed by country) to get `[p.key]` = country and `[p.path]`, then over `[p.value]` (the array of names) to get one row per player.

Every extra level of `[FLATTEN]` multiplies row counts wherever there are multiple matching child elements — this is exactly why the validation step (Section 4.4) matters: it's easy to silently duplicate rows (e.g., if `[wickets]` and `[extras]` both flatten without care, a delivery with one extra *and* one wicket could double-count runs unless the aggregation logic accounts for it).

In one sentence: `[LATERAL FLATTEN]` is how Snowflake "un-nests" JSON — turning arrays and objects buried inside a single cell into proper rows and columns you can filter, join, and aggregate like any relational table.

6. Challenges Faced

Multi-level nested JSON flattening — getting from `[innings]` down to individual `[deliveries]` required chaining five `[LATERAL FLATTEN]` calls. Getting the join order and `[outer => True]` flags right (so deliveries without extras/wickets weren't silently dropped) took careful validation.

Row explosion / double counting — every additional `[FLATTEN]` level had the potential to multiply rows when a delivery had both an extra *and* a wicket. This required deliberate use of `[CASE WHEN]` conditional aggregation in the consumption layer rather than naive `[SUM()]`, to avoid overcounting.

File size limits on the web UI stage — Snowflake's web-based stage upload tops out around 250 MB, which was far too small for ~2,411 source files. This required switching to **SnowSQL** (CLI) for bulk file upload.

Inconsistent/missing values in source JSON — fields like `[outcome.winner]`, `[event.match_number]`, and `[event.stage]` are not always present (e.g., tied matches, matches decided by no result, or matches without a numbered stage). This required defensive `[CASE WHEN ... ELSE 'NA']` logic throughout `[match_detail_clean]` rather than assuming every field is populated.

Partial load failures — a subset of the 2,411 JSON files may be malformed or structured slightly differently. Using `[ON_ERROR = CONTINUE]` in the `[COPY INTO]` statement allowed the bulk load to succeed for valid files without one bad file blocking the entire batch — though this also means load results need to be reconciled against the source file count.

Resolving natural keys to surrogate keys for the star schema — the fact table inserts require joining `[match_detail_clean]`/`[delivery_clean_tbl]` back to every dimension on natural keys (team name, venue name + city, player name). Any mismatch in spelling/formatting between the clean layer and a dimension table (e.g., a venue name that doesn't exactly match) would silently exclude that row from the fact table, so dimension population had to be carefully deduplicated (`[SELECT DISTINCT]`) before the fact load.

Designing the grain of two fact tables — deciding that `[match_fact]` needed to be at the *match* grain (with Team A/Team B columns side-by-side) while `[delivery_fact]` needed to be at the *ball* grain required separate aggregation logic and separate `[INSERT]` statements, rather than one universal fact table.

This project demonstrates a complete, production-style data engineering workflow on Snowflake: taking raw, deeply nested JSON cricket data through a disciplined **Landing → Raw → Clean → Consumption** pipeline and landing it in a proper **star schema**.

Key outcomes:

Reliable, repeatable ingestion of thousands of semi-structured JSON files using Snowflake stages, file formats, and `COPY INTO`.

Mastery of `VARIANT`/`OBJECT`/`ARRAY` handling and multi-level `LATERAL FLATTEN` to unnest complex JSON into relational tables.

A validated, constraint-enforced clean layer (primary keys, foreign keys, `NOT NULL` checks) that can be trusted by downstream consumers.

A dimensional model (`match_fact`, `delivery_fact`, and six supporting dimensions) that turns match-level and ball-by-ball cricket data into a schema ready for BI tools and ad-hoc analytics — enabling questions like "top run-scorer by match," "team performance by venue," and "toss decision impact on match outcome" to be answered with simple, fast SQL.

Overall, the project is a strong, realistic showcase of core data engineering skills — semi-structured data handling, data modeling, data quality validation, and dimensional design — applied to a domain (cricket) that closely mirrors the complexity found in real production data pipelines.